

10장 모델을 어떻게 배포하고, 무엇으로 검증하는가

10주차 · 모델 배포와 CI/CD

9장에서 champion 별칭까지 붙은 모델은 아직 실습 스크립트 안에서만 돈다. 이용자는 스크립트를 실행하지 않으므로 모델은 API 뒤에 놓여야 하고, 그 API는 어느 서버에서든 같은 형태로 실행되도록 포장되어야 한다. 이 장은 그 포장과 교체를 사람의 손이 아니라 검증 단계가 통과시키도록 만드는 과정을 다룬다.

이 장의 수치는 모두 `practice/chapter10` 을 실제로 실행해 얻은 값이다.

1. 오늘의 질문

- 예측을 미리 만들어 둘 것인가, 요청이 도착한 순간 계산할 것인가?
- 서빙 API가 "지금 무엇을 서빙하고 있는가"에 답하지 못하면 무엇을 잃는가?
- 새 모델 후보를 이용자에게 위험을 주지 않고 실제 요청 위에서 평가할 수 있는가?
- 같은 확인을 왜 `pytest`-호스트-컨테이너(container) 세 곳에서 반복하는가?

2. 설비 납품 검사로 보는 이 장

기관이 발주한 설비를 납품받아 청사에 설치하기까지의 검사 절차에 이 장의 요소가 하나씩 대응한다.

설비 납품 검사	모델 배포
도면대로 만들었는지 서류로 대조한다	<code>pytest</code> — 서버를 띄우지 않고 앱 객체를 직접 호출해 계약을 확인한다
공장에서 시운전한다	호스트 실행 스모크(smoke) — 실제 프로세스와 HTTP로 확인한다
청사의 실제 전기·배관에 연결해 다시 시운전한다	컨테이너 스모크 — 실제 배포 환경에서 확인한다
설비에 붙은 명판에서 제품명과 제조번호를 읽는다	<code>/health</code> 가 답하는 모델 이름·버전·로드 출처
새 설비를 옆에 놓고 돌려만 보되 그 출력은 쓰지 않는다	<code>shadow</code> — 후보 모델에 요청을 복제하되 응답은 쓰지 않는다

비유가 깨지는 지점은 복제 가능성이다. 설비는 같은 도면으로 만들어도 개체마다 조금씩 다르지만, 컨테이너 이미지와 그 안의 파일은 바이트 단위로 똑같은 복제본으로 옮겨진다. 그래서 배포에서는 "같은 도면으로 만들었다"에서 멈추지 않고 "바이트가 같다"까지 계산으로 확인할 수 있고, 실제로 그렇게 확인한다(6절).

3. 예측은 언제 계산되는가

모델이 있다고 예측이 이용자에게 닿는 것은 아니다. 누군가 어느 시점에 모델을 실행해야 하고, 그 시점의 선택지가 둘이다. **배치 추론**은 예측을 미리 만들어 저장해 둔다. 매일 밤 전체 지역구의 내일 민원 건수를 한꺼번에 예측해 테이블에 넣어 두면, 낮의 조회는 계산이 아니라 저장된 값을 꺼내는 일이 된다. **온라인 추론**은 요청이 도착한 순간 계산한다. 이용자가 입력을 보내면 API가 모델을 실행해 그 자리에서 답을 만든다.

어느 쪽인지는 두 질문으로 갈린다. 첫째, 요청 시점에만 알 수 있는 입력이 필요한가. 이용자가 방금 입력한 값이나 현재 위치처럼 미리 알 수 없는 입력이 있으면 배치로는 만들 수 없다. 둘째, 예측이 쓰일 지점을 미리 열거할 수 있는가. "서울 25개 자치구의 내일 예측"처럼 대상이 유한하면 미리 다 만들어 둘 수 있지만, 임의의 입력 조합이면 불가능하다.

여기서 정작 중요한 것은 **실패의 형태가 다르다**는 점이다. 배치의 실패는 조용하다. 밤 배치가 죽어도 낮의 조회는 어제 값을 아무 오류 없이 돌려주므로, 화면을 보는 담당자는 낡은 예측을 최신 예측으로 읽는다. 1장에서 본 침묵 실패가 예측 층에서 반복되는 것이다. 반대로 온라인의 실패는 시끄럽다. 서버가 죽으면 모든 요청이 즉시 실패하므로 아무도 모르고 지나갈 수는 없다. 어느 쪽이 낫다고 정해져 있지 않고, 어느 쪽의 실패인지 알고 설계하는 것이 운영의 시작이다.

정직하게 덧붙이면 일 단위 민원 건수 예측은 사실 배치로 충분하다. 대상인 자치구는 열거할 수 있고 피처인 전일 건수는 하루 한 번 확정되므로, 1분마다 새 예측을 요구할 이유가 없다. 그런데도 실습이 온라인 API를 세우는 이유는 관찰 대상 때문이다. 서빙 계약, 컨테이너 포장, 검증 게이트라는 **배포의 규율은 온라인 서빙에서 전부 한꺼번에 등장한다**. 거꾸로 배치로 충분한 문제에 상시 API를 세우면 비용과 장애 지점을 스스로 늘리는 과잉 설계가 되므로, 발주 문서에서 "실시간 AI"라는 표현을 보면 요청 시점에만 아는 입력이 실제로 있는지부터 물어야 한다.

4. 서빙 API는 모델에 신원과 계약을 입힌다

API는 모델을 감싸는 껍데기가 아니라 모델이 지켜야 할 계약이다. 실습의 서빙 앱은 그 계약을 세 가지로 나눠 선언한다.

신원 응답. GET /health 는 살아 있지만 답하지 않고 **무엇을 서빙 중인지**를 함께 답한다. 실측에서 호스트 실행은 `model_name=complaint_daily_forecaster`, `model_version=1`, `model_source=registry-alias` 를 돌려준다. 모델 버전 1은 9장에서 champion으로 승격된 v1이고, 로드 출처가 레지스트리(registry) 별칭 조회였다는 뜻이다.

신원이 실린 예측. POST /predict 는 자치구 코드와 전일 건수를 받아 예측값과 모델 신원을 함께 돌려준다. 강남구 코드 11680에 전일 건수 9를 보내면 응답은 **6.0**이다. 입력 9가 응답에 반영되지 않는 것은 오류가 아니라 champion인 v1이 평균 예측기이기 때문이며, 훈련 y의 평균 $36/6 = 6.0$ 이 손으로 계산된다.

입력 검증. 전일 건수에 -1을 보내면 모델에 닿기 전에 **422**로 거절된다. 자치구 코드는 5자리 숫자, 전일 건수는 0 이상 정수라는 선언을 pydantic이 스키마 계약으로 집행하기 때문이다. 잘못된 입력이 모델까지 흘러가 그럴듯한 숫자로 되돌아오는 경로를 막는 것이 이 계약의 목적이다.

신원 응답 한 줄이 왜 계약에 들어가는지는 감사 상황에서 드러난다. 콜센터 상담 지원 시스템이 "이 예측의 근거를 소명하라"는 민원을 받았다고 하자. 응답에 모델 이름과 버전이 실려 있으면 그 시각 그 답을 만든 모델이 특정되고, 9장 레지스트리를 따라 실행 기록·훈련 데이터 지문·파라미터까지 거슬러 올라갈 수 있다. 응답에 예측값만 있으면 그 소급은 "그때 아마 이 버전이었을 것"이라는 추정에서 멈춘다. **응답에 신원을 실는 한 줄이 감사 가능성의 분기점이다.**

모델은 요청마다 로드하지 않고 기동 시점에 한 번만 로드한다. 요청마다 로드하면 로드 시간과 레지스트리 장애가 요청 경로 안으로 들어오기 때문이다. 무겁고 위험한 일은 기동 시점에 몰고 요청 경로에는 계산만 남기는 것인데, 여기에 대가가 하나 붙는다. 이미 떠 있는 프로세스는 별칭이 옮겨진 사실을 모른다. 로드 코드가 `models:/complaint_daily_forecaster@champion` 처럼 버전 번호가 아닌 별칭을 가리키므로 승격 때 코드는 한 줄도 바뀌지 않지만, 그 대신 다시 기동하기 전까지는 옛 모델을 계속 서빙한다. 다음 절의 두 층 문제가 여기서 나온다.

5. 교체는 두 층에서 일어나고, 되돌림에는 기준이 필요하다

새 버전이 옳다는 확신은 배포 전에 완전해지지 않는다. 그래서 배포 전략은 전부 한 질문의 답이다 — **틀렸을 때 피해가 어디까지 번지는가(실패 반경, blast radius)를 어떻게 제한하는가.** 전량을 한 번에 교체하면 반경은 100%이고, 새 버전에 트래픽 일부만 흘리는 카나리(canary)는 그 비율만큼으로 반경을 줄인다.

실습이 고른 것은 **shadow**다. 실제 요청을 후보 모델에도 복제해 보내되 이용자에게 돌려주는 응답은 champion 것만 쓰므로, 후보가 무엇을 답하든 이용자에게 닿지 않는다. 실패 반경이 0이다. 대가는 계산 자원이 두 배로 들고 "이용자가 그 답에 어떻게 반응하는가"는 관찰할 수 없다는 점이다. 공공에서 이 전략이 특히 맞는 이유가 하나 더 있다. 트래픽을 나눠 보내면 같은 시각에 시민마다 다른 모델이 응답하게 되는데, 민원 배정이나 복지 판정처럼 권리에 영향을 주는 결정에서는 그 중간 상태 자체가 형평성 문제가 된다.

실측을 보면 shadow가 무엇을 만들어 내는지 분명해진다. 강남구에 전일 건수 9를 보냈을 때 이용자가 받은 값은 champion의 **6.0**이고, 로그에만 남은 challenger v2의 값은 **7.3636**이다. 9장 회귀 계수로 $3.6818 + 0.4091 \times 9 = 81/11$ 을 손으로 계산할 수 있다. 두 값의 차이 1.36이 곧 관찰 자료이며, 이용자 위험을 0으로 둔 채 반려된 후보를 실요청 위에서 재평가할 근거가 된다.

교체를 실제로 하려면 층이 둘이라는 점을 알아야 한다. **모델 층**의 교체는 레지스트리에서 champion 별칭이 v1에서 v2로 옮겨지는 일이고, **서비스 층**의 교체는 컨테이너를 다시 기동해 트래픽을 새 실물로 넘기는 일이다. 앱이 기동 시점에 모델을 한 번만 로드하므로(4절) 별칭만 옮겨서는 서빙에 반영되지

않는다. 두 층이 다 움직여야 교체가 완성되고, 되돌릴 때도 별칭을 되돌리고 이전 이미지를 다시 기동하는 두 동작이 필요하다.

되돌리는 **방법**을 아는 것과 **언제** 되돌릴지 정해 두는 것은 다른 일이다. 사고가 커지는 자리는 대개 경로가 없어서가 아니라 기준이 없어서다. 지표가 나빠지는 동안 "조금 더 보자"가 반복되고, 되돌림은 피해가 다 번진 뒤에 온다. 그래서 롤백(rollback) 트리거는 **어떤 지표를, 어느 임계에서, 얼마의 관측 창으로** 판정할지 세 조각을 함께 적어야 한다. 셋 중 하나라도 비어 있으면 되돌릴 시점이 정해지지 않으므로 트리거로 작동하지 않는다. 이 장 범위에서 바로 쓸 수 있는 트리거는 배포 직후 스모크다 — 입력 9에 6.0이 나오지 않으면 이전 이미지로 되돌린다. 임계와 관측 창은 되돌림의 대가에서 역산한다. 이전 이미지를 다시 띄우는 정도로 끝나면 창을 짧게 잡아 빨리 되돌리는 편이 싸고, 저장소 스키마 변경까지 따라오면 창을 늘려 오탐을 줄이는 편이 낫다.

6. 검증은 층으로 쌓고, 동일성은 바이트로 확인한다

이 실습을 만들면서 실제로 겪은 오류가 층의 필요성을 그대로 보여 준다. 서빙 앱은 설정 기본값을 자기 파일 위치에서 두 단계 위 디렉터리 기준으로 계산했다. pytest가 통과했고 호스트 실행도 통과했는데, 컨테이너에서는 `/health` 가 끝내 응답하지 않았다. 이미지 안에서 앱은 `/app/app.py` 에 놓이므로 "두 단계 위"가 존재하지 않았고, 경로 계산이 import 시점에 `IndexError`로 실패한 것이다.

주목할 것은 수정 내용이 아니라 실패의 구조다. 앞의 두 게이트는 이 결함을 **원리적으로** 잡을 수 없었다. 둘 다 저장소 배치 위에서 실행되므로 그 경로 가정이 항상 참이었고, 가정이 깨지는 유일한 장소가 컨테이너였기 때문이다. 실행 환경이 달라야만 드러나는 결함이 존재한다는 것, 그래서 컨테이너 스모크가 생략 가능한 중복이 아니라 독립 게이트라는 것이 이 사고의 교훈이다.

그래서 실습의 게이트는 세 층이고 각 층이 잡는 결함이 다르다.

- **pytest**는 서버 프로세스 없이 앱 객체를 직접 호출해 신원 누락·검증 누락 같은 계약 위반을 초 단위에 잡는다. 실측 4건 통과.
- **호스트 실행 스모크**는 실제 프로세스와 HTTP를 쓰므로 기동 절차·포트·레지스트리 연결의 결함을 잡는다.
- **컨테이너 스모크**는 실제 배포 환경이므로 이미지 배치·의존성 누락·경로 가정의 결함을 잡는다.

층의 마지막 질문이 **환경 동등성**이다. 같은 입력에 호스트와 컨테이너가 같은 답을 내는가. 실측은 호스트 6.0과 컨테이너 6.0이 같고 두 환경 모두 잘못된 입력을 422로 거절한다. 다만 같은 답을 냈다는 것과 그 답의 출처를 증명하는 방식이 같다는 것은 별개다. 호스트는 `model_source=registry-alias` 로 레지스트리에 조회해 신원을 답하고, 컨테이너는 `model_source=local-dir` 로 이미지에 함께 구워 넣은 `export_info.json` 을 읽어 답한다. 검수는 "같은 답이 나오는가"와 "무엇을 서빙 중인지 답할 수 있는가"를 둘 다 묻는다.

컨테이너 쪽 출처가 파일이 될 수 있는 이유는 이미지가 코드와 의존성 버전과 **모델 실물까지** 하나의 불변 단위로 포장하기 때문이다. 9장 재현 4요소 중 "환경"이 문서가 아니라 실행 가능한 실물로 고정되고, 그 결과 이미지가 곧 롤백 단위가 된다. 빌드 절차는 champion 실물을 내려받는 데서 그치지 않고 내려받은

모델을 로드해 예측까지 시켜 본 뒤(`export_check_forecast` = 6.0) 어느 별칭의 몇 번 버전이었는지를 `export_info.json` 으로 동봉한다. 쓰기가 성공했다는 사실을 믿지 않고 읽어서 확인하는 9장의 규율이 그대로 이어진 것이다.

여기서 이 장의 마지막 질문이 나온다. "같은 코드로 만들었다"와 "바이트가 동일하다"는 다른 진술이다. 앞의 진술은 같은 소스에서 출발했다는 주장일 뿐이어서, 옮겨 붙이는 과정에서 한 줄이 편집되거나 줄바꿈·인코딩이 바뀌는 변형을 배제하지 못한다. 뒤의 진술은 파일의 내용을 실제로 해시로 계산해 대조한 결과이므로 그런 변형을 전부 걸러 낸다. 14장의 통합 시스템은 이 장의 `app.py` 를 복사해 쓰면서 원본과 sha256을 대조하고, 그 결과를 검수표 항목으로 남긴다. 물론 바이트 동일성이 그 파일이 옳게 동작한다는 것까지 보증하지는 않는다. 그래서 검수는 바이트 동일성과 스모크(입력 9 → 6.0)를 함께 확인한다.

이 게이트들을 매번 실행하는 주체를 사람에서 저장소로 옮긴 것이 CI(Continuous Integration, 지속적 통합)다. CI는 새 검증을 더하는 장치가 아니다. 게이트는 이미 다 있고, 바뀌는 것은 "한 줄 고쳤는데 뭘"이라며 건너뛴 선택지가 구조적으로 사라진다는 점뿐이다. 반대로 CI가 잡지 못하는 것도 분명하다. 데이터가 변해 모델이 낡아 가는 드리프트는 11장의 일이고 부하에서의 성능은 실무교재 `docs/chapter12.md` 가 다루므로, **CI 통과는 배포 가능한 필요조건이지 충분조건이 아니다.**

7. 실습 10.1 모델 API 컨테이너 빌드와 호출

실습 목표

9장 champion 별칭을 FastAPI 서빙이 로드하게 하고, 같은 API를 `pytest` → 호스트 → 컨테이너의 세 층으로 검증한다. shadow 로그로 challenger를 관찰하고, 호스트와 컨테이너가 같은 입력에 같은 예측을 내는지 실측으로 대조한다.

실행 환경

Python 3.10 이상(3.13에서 검증)과 Docker 데몬(28.x에서 검증)이 필요하다. Docker가 없으면 `--skip-docker` 로 시나리오 1~3까지 완주할 수 있다.

시작 전 준비

```
python scripts/setup_practice.py 10
```

Docker는 선택이다. 없으면 스크립트가 주의로만 알리고, 실습은 `--skip-docker` 로 1~3단계까지 진행한다. 4단계 컨테이너 검증까지 하려면 Docker Desktop을 실행한다.

실행 명령

```
cd practice/chapter10
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter10.py # Docker 없이: python run_chapter10.py --skip-docker
```

스크립트는 네 시나리오를 순서대로 돌린다. ① bootstrap이 9장과 같은 데이터로 레지스트리를 재구성하고 champion을 내보낸다. ② pytest로 서빙 계약을 검증한다. ③ 호스트에서 8010 포트로 앱을 띄워 호출하고 shadow 로그를 관찰한다. ④ 이미지를 빌드해 8011 포트로 띄우고 같은 호출을 반복해 환경 동등성을 확인한다.

실행 전에 예측을 적는다

실행하기 전에 아래 표의 "내 예측" 칸을 채우고, 실행한 뒤 `ch10_deploy_report.json` 의 `consistency` · `host` · `container` 값으로 "실측" 칸을 채운다.

질문	내 예측	실측
두 환경의 예측값이 같겠는가		호스트 6.0 = 컨테이너 6.0
/health 의 <code>model_source</code> 가 두 환경에서 같겠는가		호스트 registry-alias / 컨테이너 local-dir
잘못된 입력의 응답 코드는 무엇이겠는가		두 환경 모두 422

핵심 코드

```
model = mlflow.pyfunc.load_model(uri) # 컨테이너: uri="/app/model"
info = json.loads(info_path.read_text(encoding="utf-8"))
return model, {"model_name": info["model_name"],
               "model_version": str(info["champion_version"]),
               "model_source": "local-dir"}
```

전체 코드는 `practice/chapter10/code/10-1-model-api/` 참고
(`app.py`·`bootstrap_registry.py`·`test_app.py`·`Dockerfile`)

실행 결과 확인 사항

1. 시나리오 1: 훈련 데이터 지문 `96f6b6b3ce9dcdd1...` 이 출력되고(9장과 같은 데이터라는 증명), export 로드 검증 예측이 6.0으로 나온다. champion=v1(train_mae 1.0), challenger=v2(train_mae 1.0455).

2. **시나리오 2:** pytest 4 passed.
3. **시나리오 3:** /health 가 model_version=1, model_source=registry-alias, shadow_enabled=true를 답한다. 전일 9 → 6.0, 잘못된 입력 → 422. shadow 관찰은 champion 6.0 / challenger 7.3636.
4. **시나리오 4:** /health 의 model_source가 local-dir로 바뀌고 예측은 6.0으로 호스트와 같다.
5. **최종 출력:** CH10_RUN_PASS (Docker 미사용 시 CH10_RUN_PARTIAL).

실측과 대조하기

예측값이 같은 것은 우연이 아니라 세 가지를 함께 고정했기 때문이다. 모델 실물을 이미지에 구웠고, 의존성 버전을 requirements.txt 로 고정했고, 입력 계약을 pydantic 스키마로 못 박았다. 셋 중 하나만 흔들려도 값은 갈릴 수 있으므로 "같은 코드니까 같겠지"는 근거가 되지 못한다. 코드는 셋 중 하나일 뿐이다.

"다르다"로 예측했다면 그때 떠올린 걱정이 그대로 실제 위험 목록이다. 이미지와 호스트의 의존성 버전이 갈리는 경우, 빌드 시점의 champion과 지금 별칭이 가리키는 버전이 어긋나는 경우가 대표적이며, 이번 실행에서 나타나지 않은 이유는 그것을 막는 장치를 넣어 두었기 때문이지 일어나지 않는 일이어서가 아니다.

한 가지 더 놓치기 쉽다. 환경 동등성은 **두 환경이 모두 떴을 때만** 물을 수 있는 질문이다. 6절의 경로 가정 결함은 값이 갈리는 실패가 아니라 컨테이너가 아예 응답하지 못하는 실패였다. 예측표를 값 비교로만 채웠다면 그 실패 유형을 통째로 빠뜨린 것이다.

산출물

- ch10_deploy_report.json — bootstrap·테스트·호스트·컨테이너·shadow 실측의 증거 파일
- ch10_release_checklist.json — 배포 전 검수 체크리스트 8항목(실행 결과에서 자동 생성, 실측 전항 통과)
- ch10_bootstrap_summary.json — 승격·반려 판정과 export 출처
- ch10_shadow_log.jsonl — 요청별 champion/challenger 관찰 로그

체크리스트가 손으로 쓰는 문서가 아니라 실행 결과에서 계산되어 나온다는 점이 중요하다. 문서가 실물에서 나오면 문서와 실물이 어긋날 수 없다.

8. 핵심 요약

1. 배치와 온라인은 속도가 아니라 실패의 형태로 갈린다. 배치는 밤 작업이 죽어도 낮의 조치가 어제 값을 오류 없이 돌려주는 조용한 실패를 만들고, 온라인은 서버가 죽으면 모든 요청이 즉시 실패하는 시끄러운 실패를 만든다. 어느 쪽이 낫다고 정해져 있지 않고, 어느 쪽인지 알고 설계해야 한다.

2. 서빙 API의 최소 계약은 신원·예측·검증 셋이다. `/health` 가 모델 이름과 버전과 로드 출처를 담하고, `/predict` 가 예측에 신원을 실어 보내고, 스키마에 어긋난 입력을 422로 거절한다. 신원이 빠지면 "그때 아마 그 버전이었을 것"이라는 추정만 남고 감사에 답할 수 없다.

3. shadow는 실패 반경을 0으로 두고 후보를 재평가한다. 이용자는 champion의 6.0만 받고 challenger의 7.3636은 로그에만 남으므로, 시민마다 다른 모델이 응답하는 상태를 만들지 않고도 후보를 실요청 위에서 관찰한다. 다만 교체는 별칭 이동(모델 층)과 재기동(서비스 층) 두 층이 다 움직여야 완성되고, 되돌림에는 지표·임계·관측 창 세 조각의 기준이 필요하다.

4. 층마다 잡는 결함이 다르므로 검증은 겹쳐야 한다. 경로 가정 결함은 pytest와 호스트 실행을 원리적으로 통과하고 컨테이너에서만 드러났다. 실행 환경이 달라야만 나타나는 결함이 있으므로 컨테이너 스모크는 중복이 아니라 독립 게이트다.

5. "같은 코드"가 아니라 "같은 바이트"를 확인한다. 같은 소스에서 출발했다는 주장은 옮기는 과정의 변형을 배제하지 못하지만, sha256을 계산해 대조하면 그 변형이 전부 걸러진다. 14장이 이 장의 `app.py` 를 복사해 쓰면서 해시를 대조하는 이유가 이것이며, 바이트 동일성은 동작의 정확성까지는 보증하지 않으므로 스모크(9 → 6.0)와 함께 본다.

9. 과제

실습을 실행해 생성된 다음 파일을 LMS에 그대로 업로드한다.

- `practice/chapter10/data/output/ch10_deploy_report.json`

이어서 본인 산출물의 값을 인용해 세 가지를 각각 세 문장 이내로 쓴다.

1. 본인 파일의 `consistency.host_forecast` 와 `consistency.container_forecast` 를 적고, 두 값이 같다는 사실이 무엇을 보증하고 무엇을 보증하지 못하는지 구분해 쓴다.
2. 본인 파일의 `host.health.model_source` 와 `container.health.model_source` 를 나란히 적고, 같은 답을 냈는데도 출처를 증명하는 방식이 다른 이유를 쓴다.
3. 서빙 앱을 다른 장으로 옮겨 쓸 때 "같은 코드로 만들었다"가 아니라 "바이트가 동일하다"를 확인하는 이유를 쓴다. 두 진술이 각각 무엇을 보증하는지가 드러나야 한다.

수업일 포함 7일 이내에 제출한다.

10. 더 알아보기

이 장에서 다루지 않은 내용은 실무교재 `docs/chapter10.md` 에 있다.

- 네 배포 전략의 비교표 — 일괄·blue-green·카나리·shadow를 실패 반경·롤백 경로·추가 비용으로 정리한 표와, 민간의 트래픽 분할을 공공에서 변형하는 방식: `docs/chapter10.md` §10.2
- 기동 대기 시간을 얼마로 잡는가 — 실습의 90초 헬스체크 대기과, "중단"과 "지연"을 구분하는 생존 확인 판정: §10.4

- **CI 워크플로의 단계 구성과 게이트 배치** — GitHub Actions 워크플로 정의, 게이트를 어느 층에 둘지 정하는 세 질문, 배포 전 검수 체크리스트 8항목의 근거: §10.5
- **배포 설계표 작성 실습** — 요구사항에서 배포 전략·롤백 트리거·게이트 배치를 직접 고르고 근거를 적는 1단계 설계 과제: docs/chapter10.md 실습 10.1의 1단계와 4단계